Session 4 · Part Two
FastAPI

From a website to an API

Information Systems Development (SE)

Today in four parts

01 What an API is
Why your Flask app only talks to humans

02 FastAPI basics
Routes, parameters, and free documentation

03 Data in and out
Pydantic models, POST, and validation

04 Serving real data
The JSON file from part one, on the web

Where we are

This morning

- You wrote a catalogue to a JSON file.
- It survives a restart.

This hour

- Anything on the internet can ask for that catalogue
- and get an answer a machine understands.

01

What an API is

Your program, talking to other programs

Your Flask app has one audience

- Flask returns HTML. HTML is for eyes — headings, colours, layout.

- A phone app cannot read it. Another company's server cannot read it.
- A data scientist loading your prices into Python cannot read it either.

- They all want the same thing: the data, with no decoration.

Two answers to the same question

HTML — for people

- Laptop — 899.0 EUR

- Carries layout. A machine has to guess which part is the price.

JSON — for programs

- {"name": "Laptop", "price": 899.0}

- Carries meaning. The price is a number, labelled price.

Who actually calls an API

A mobile app
Same data, a screen you did not build

Another service
Your warehouse asking your shop what is in stock

A front end
React or Vue, fetching and drawing the page itself

A data scientist
Three lines of Python, straight into a dataframe

The same data, two ways

```python
# Flask -- returns HTML for a human
@app.route("/products")
def products():
    return render_template("index.html", rows=catalogue)


# FastAPI -- returns JSON for a program
@app.get("/products")
def products():
    return catalogue
```

Look closely

The second one has no template and no HTML.

You return the Python list, and FastAPI turns it into JSON for you.

That is most of the difference.

02

FastAPI basics

Everything you know from Flask, plus types

Flask or FastAPI?

Flask

- Older, smaller, everywhere.
- Built for returning HTML pages.
- You write your own validation and your own docs.

FastAPI

- Newer, built for APIs.
- Returns JSON by default.
- Reads your type hints, then validates and documents for free.

Hello, API

```
# pip install "fastapi[standard]"

# main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Hello from MiniShop"}

# in the terminal:
#   fastapi dev main.py
```

Three differences

@app.get instead of @app.route — the method is in the name.

You return a dictionary, not a string.

You do not run the file. A server called uvicorn runs it for you.

The part that wins people over

- Start the server, then open `/docs` in your browser.
- FastAPI has read your code and built a documentation page: every route, every parameter, every field — with a button to try each one live.
- You wrote none of it. Nobody has to keep it up to date.
- There is a second page at `/redoc` if you prefer it, and the raw specification sits at `/openapi.json` .

What you can return

```
@app.get("/one")
def one():
    return {"name": "Laptop", "price": 899.0}

@app.get("/many")
def many():
    return [{"name": "Laptop"}, {"name": "Mouse"}]

@app.get("/count")
def count():
    return {"total": len(catalogue)}
```

The rule

Return a dict, a list, a string or a number.

FastAPI converts it to JSON on the way out.

Your list of dictionaries needs no conversion at all.

Part of the URL: path parameters

```python
@app.get("/products/{name}")
def one_product(name: str):
    return {"asked_for": name}

# GET /products/laptop
# -> {"asked_for": "laptop"}


@app.get("/orders/{order_id}")
def one_order(order_id: int):
    return {"id": order_id}

# GET /orders/abc  -> 422, a clear error. Not a crash.
```

Note the types

The name in {braces} must match the parameter name.

order_id: int tells FastAPI to convert and check it.

Bad input is rejected before your function ever runs.

After the question mark: query parameters

```python
@app.get("/search")
def search(q: str, limit: int = 10):
    hits = [r for r in catalogue if q in r["name"].lower()]
    return hits[:limit]

# GET /search?q=lap            -> up to 10 hits
# GET /search?q=lap&limit=3    -> up to 3
# GET /search                  -> 422, q is required
```

How it decides

A parameter named in the path is a path parameter.

Any other parameter becomes a query parameter.

Give it a default and it is optional. No default means required.

Why the type hints matter

Without them

- Everything arrives as text.
- You convert by hand, you validate by hand, and you write the docs by hand.

With them

- FastAPI converts, validates, returns a clear error, and documents the route — from the same one line.

03

Data in and out

Describing what you accept

A Pydantic model is a shape

```python
from pydantic import BaseModel

class Product(BaseModel):
    name: str
    price: float
    stock: int = 0

# a class, like Session 5 -- but you only
# describe the fields. Pydantic does the rest.
```

What it buys you

One place that says what a product is.

Anything that does not fit is refused, with a message naming the field.

The /docs page shows the shape to whoever calls you.

Receiving data: POST

```
@app.post("/products")
def add_product(product: Product):
    catalogue.append(product.model_dump())
    save(catalogue)
    return {"added": product.name}

# the body sent by the client:
# {"name": "Hub", "price": 149.0, "stock": 7}
```

Read the signature

@app.post, because this changes something.

product: Product is how FastAPI knows to read the request body.

.model_dump() turns the model back into a plain dictionary.

save() is your function from part one.

GET or POST?

GET

- Asks for something and changes nothing.
- Safe to repeat. You can type it into a browser.
- Parameters live in the URL.

POST

- Sends something and changes the server.
- Repeating it adds the product twice.
- Data lives in the body, not the URL.

04

Serving real data

The file from part one, on the web

Load the catalogue you saved this morning

```
import json
from pathlib import Path
from fastapi import FastAPI

FILE = Path("catalogue.json")
app = FastAPI(title="MiniShop API")

def load():
    try:
        return json.loads(FILE.read_text())
    except FileNotFoundError:
        return []

catalogue = load()
```

Nothing new here

json, pathlib and try/except are all from part one.

title= shows up as the heading on your /docs page.

Loading once at start-up is fine for a classroom. A real app uses a database.

Two routes over one list

```
@app.get("/products")
def all_products():
    return catalogue


@app.get("/products/{name}")
def one_product(name: str):
    for row in catalogue:
        if row["name"].lower() == name.lower():
            return row
    return {"error": "not found"}
```

Good, but

The second route answers 200 OK even when nothing was found.

A client cannot tell success from failure.

The next slide fixes that properly.

Failing honestly

```python
from fastapi import FastAPI, HTTPException

@app.get("/products/{name}")
def one_product(name: str):
    for row in catalogue:
        if row["name"].lower() == name.lower():
            return row
    raise HTTPException(status_code=404,
                        detail=f"No product called {name}")
```

Status codes

200 — here it is.

404 — I understood you, but there is no such thing.

422 — your request was the wrong shape. FastAPI sends this one for you.

500 — I broke. Your bug, not theirs.

A filter, and a summary

```python
@app.get("/in-stock")
def in_stock():
    return [r for r in catalogue if r["stock"] > 0]


@app.get("/stats")
def stats():
    total = sum(r["price"] * r["stock"] for r in catalogue)
    return {
        "products": len(catalogue),
        "stock_value": round(total, 2),
    }
```

This is the job

Each route answers one question about the data.

Same functions you wrote in Session 3 — now anyone can call them.

Open /docs and try them without writing a line of client code.

So which one do I use?

Reach for Flask

- The answer is a page a person looks at.
- A shop, a blog, a dashboard with templates.

Reach for FastAPI

- The answer is data another program consumes.
- A mobile back end, a service, anything with a separate front end.

Thank you

Dr. Pejman Ebrahimi
pejman.ebrahimi@uni.li
University of Liechtenstein